

Branch Prediction Meets Token Prediction: OS Speculation Principles in Modern LLM Inference

Kuang Wang

Student ID: 224040348

Course Project Report Submitted to
Advanced Operating Systems

School of Data Science
The Chinese University of Hong Kong, Shenzhen

May 2026

Abstract

Autoregressive decoding remains the dominant inference paradigm for sequence generation, yet its token-by-token dependency creates a severe serial bottleneck even on highly parallel hardware. This paper studies one influential acceleration line, *speculative decoding*, through three representative papers: *Blockwise Parallel Decoding for Deep Autoregressive Models* (Stern et al., 2018), *Speculative Decoding: Exploiting Speculative Execution for Accelerating Seq2seq Generation* (Xia et al., 2023), and *EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty* (Li et al., 2024). Rather than treating them as isolated techniques, this paper organizes them as a coherent evolution in speculative execution for neural generation and explicitly relates them to the operating-systems notion of branch prediction.

The central claim is that speculative decoding replays, at the LLM inference level, the same systems logic used by branch prediction to reduce control-hazard stalls: use a cheap predictor to keep an expensive sequential pipeline busy, then validate and either commit or roll back. Under this view, the three selected papers solve three successive bottlenecks. Blockwise Decoding establishes the earliest practical *draft–verify–accept* skeleton and preserves greedy equivalence through longest-prefix validation. SpecDec turns speculation into a stronger systems recipe for seq2seq generation by redesigning both the drafter and the verification rule, thereby improving the speed–quality frontier without replacing the original autoregressive model. EAGLE then shifts the drafting space from tokens to hidden features and shows that the real difficulty of LLM-scale speculation lies not only in prediction accuracy but also in *feature uncertainty*. By conditioning feature prediction on shifted tokens and reusing the target model’s language head, EAGLE achieves stronger acceptance behavior while preserving the target distribution during verification.

The main conclusion is that speculative decoding should be understood not as one fixed algorithm but as a design space defined by four coupled choices: what is speculated, how speculation is verified, what correctness guarantee is preserved, and how the method maps to real hardware efficiency. From an Advanced Operating Systems perspective, its importance lies in converting a logically sequential generation loop into a controlled speculative pipeline with predictor cost, rollback penalty, verifier amortization, and throughput–latency trade-offs analogous to those in classical processor speculation. In this sense, the governing narrative of the report is precisely *branch prediction meets token prediction*.

Chinese Abstract

自回归生成虽然在语言模型与序列模型推理中占据主导地位，但其逐词元生成的执行方式会在推理阶段形成严重的串行瓶颈。本文以“分支预测契合词元预测”为总主线，围绕投机解码这一加速范式，对三篇代表性论文进行系统化课程论文式分析：面向深度自回归模型的分块并行解码、投机解码：利用投机执行加速序列到序列生成与EAGLE：投机采样需要重新思考特征不确定性。本文不将三者视为彼此孤立的方法，而是将其组织为一条从操作系统与计算机体系结构中的经典投机执行原理，逐步演进到现代大模型推理加速机制的研究链条。

本文的核心观点是：投机解码在大语言模型推理层面上，重现了分支预测用于降低控制冒险停顿的系统思想，即用一个轻量化预测器提前探索未来执行路径，再由权威执行结果进行验证、提交或回滚。从这一视角看，三篇论文分别解决了投机解码演化中的三个关键瓶颈。2018年的分块并行解码给出了最早清晰的“草稿-校验-采纳”框架，并通过最长前缀校验保证贪心解码等价输出；2023年的投机解码则针对序列到序列场景重新设计了草稿器与校验机制，使投机解码从早期原型发展为更具实用性的系统方案；2024年的EAGLE进一步指出，词元层级草稿生成本身就是大模型场景下的困难点，因此将投机对象转移到变化更平滑的特征空间，并通过偏移词元消除特征不确定性，在保持目标模型输出分布不变的前提下获得更强的加速效果。

全文的最终结论是：投机解码不是单一算法，而是由“投机对象”、“验证机制”、“正确性保证”与“系统映射方式”四个维度共同定义的设计空间。从高级操作系统视角看，其价值不只是“让模型更快”，而在于把逻辑上串行的生成过程，重构为具有预测器开销、回滚代价、验证摊销与吞吐-延迟权衡的受控投机流水线。换言之，这一主题的真正意义在于：经典分支预测原理正在现代大语言模型推理中以词元预测的形式重新出现。

Acknowledgments

This paper is written in accordance with the course report guidelines for Advanced Operating Systems and builds upon the three papers introduced for the topic of speculative decoding.

Contents

Abstract	i
Chinese Abstract	ii
Acknowledgments	iii
1 Introduction	1
1.1 Problem statement	1
1.2 Motivation from operating systems and computer architecture	1
1.3 Why this topic fits Advanced Operating Systems	2
1.4 From branch prediction to speculative decoding	2
1.5 Research objective and central thesis	3
1.6 Contributions of this course paper	4
2 Background, Related Work, and Evaluation Framework	5
2.1 A unified draft–verify–commit view	5
2.2 Speculation as controlled rollback	5
2.3 Correctness regimes	6
2.4 Latency model and acceptance horizon	6
2.5 Evaluation dimensions	6
3 Paper I: Blockwise Parallel Decoding for Deep Autoregressive Models	8
3.1 Motivation and historical significance	8
3.2 Method formulation	8
3.3 Framework and architectural realization	9
3.4 Quantitative evidence	10
3.5 Strengths, limitations, and insight	11

4	Paper II: Speculative Decoding for Accelerating Seq2seq Generation	12
4.1	Why Blockwise Decoding was not enough	12
4.2	Spec-Drafter as a dedicated proposal model	12
4.3	Relaxed Spec-Verification	14
4.4	Why the paper is practically important	14
4.5	Experimental analysis	15
4.6	System interpretation and limits	16
5	Paper III: EAGLE and Feature-Level Speculative Sampling	18
5.1	Motivation: token prediction may be the wrong interface	18
5.2	Feature uncertainty as the real bottleneck	18
5.3	Method: shifted tokens plus feature autoregression	19
5.4	Framework and verification pipeline	19
5.5	Quantitative results	20
5.6	Strengths and limitations	22
6	Cross-Paper Comparison and Mechanism-Level Discussion	23
6.1	One topic, three bottlenecks	23
6.2	Comparative summary	23
6.3	How the drafting interface evolves	24
6.4	Correctness, rollback, and efficiency	24
6.5	Which paper contributes what to the final understanding	25
6.6	Open questions	25
7	Conclusion	26

List of Figures

3.1	Blockwise parallel decoding pipeline: predict a block of future tokens, verify them with the base model, and commit only the longest valid prefix.	9
3.2	Combined scoring and proposal reduces the overhead of running separate proposal and verification passes.	9
3.3	Transformer realization of Blockwise Decoding using shared backbone features and multiple future-token heads.	10
3.4	Wall-clock speedup is non-monotonic in the block size because larger horizons increase both prediction difficulty and overhead.	11
4.1	SpecDec replaces head-based future prediction with a dedicated drafter optimized for the block drafting task itself.	13
4.2	Distinct attention patterns across drafted positions motivate a specialized SpecDrafter rather than a shared-head extension.	13
4.3	Spec-Verification accepts drafted tokens that are sufficiently competitive under the verifier, rather than insisting on exact top-1 equality.	14
4.4	SpecDec improves both latency and throughput, but the operating point still depends on drafter cost and acceptance behavior.	16
5.1	Feature uncertainty: future hidden features are not uniquely determined by past features alone because the next token decision changes the latent trajectory. . .	18
5.2	Method comparison: EAGLE differs from classical token drafting by predicting future features while using shifted tokens to resolve uncertainty.	19
5.3	EAGLE pipeline: feature drafting, LM-head token generation, draft-tree expansion, and tree-attention verification by the target LLM.	20
5.4	Greedy-decoding speedup reported for EAGLE across multiple LLMs.	20
5.5	Input ablation in EAGLE: shifted-token conditioning materially improves acceptance and speed by resolving feature uncertainty.	21

List of Tables

1.1	Formal correspondence between CPU branch prediction and LLM speculative decoding.	3
3.1	Representative machine-translation results from Blockwise Decoding. Each cell reports BLEU / mean accepted block size.	10
3.2	Iteration reduction and wall-clock speedup reported by Blockwise Decoding.	10
4.1	Main translation results of SpecDec versus baselines. Each cell reports speedup / BLEU.	15
4.2	Drafter ablation in SpecDec.	15
4.3	Verification trade-off in SpecDec. Each cell reports Tok. / BLEU, with speed shown in parentheses when provided.	16
5.1	Representative EAGLE results reported in the analysis notes.	21
6.1	Cross-paper comparison of mechanisms, guarantees, and bottlenecks.	23

Chapter 1

Introduction

1.1 Problem statement

Autoregressive generation is the standard decoding mode for machine translation, summarization, code generation, and large language model (LLM) serving, but it also creates a token-level serial dependency that has been widely recognized as a central inference bottleneck (Leviathan et al., 2023; Chen et al., 2023; Zhou et al., 2024). Its probabilistic form is

$$p(y_{1:T} \mid x) = \prod_{t=1}^T p(y_t \mid y_{<t}, x). \quad (1.1)$$

Equation (1.1) implies that output token y_t cannot be committed until the prefix $y_{<t}$ is known. Even if each forward pass contains substantial matrix-level parallelism, the *outer loop* of decoding remains serial. This mismatch is the fundamental latency bottleneck studied in this paper.

1.2 Motivation from operating systems and computer architecture

The motivation of this topic can be stated most clearly through the branch-prediction analogy used in the course presentation. In a pipelined processor, a control-transfer instruction introduces a *control hazard*: the machine does not yet know which instruction stream should be fetched next (Hennessy and Patterson, 2019). If the processor waits for full resolution, bubbles appear in the pipeline and throughput drops. Branch prediction avoids this stall by *speculating* on the future path, continuing execution early, and later either committing correct work or flushing mispredicted work.

Autoregressive generation suffers from an analogous stall pattern. The target model can process the current state with heavy internal parallelism, but it cannot *commit* token y_{t+1} before token prefix $y_{\leq t}$ has been fixed. As a result, decoding behaves like a sequential

control dependency stretched across the whole generation loop. The natural systems question is therefore the same as in branch prediction:

can we speculate on future work cheaply, keep the expensive pipeline busy, and recover safely when the guess is wrong?

Speculative decoding answers this question with a draft model, a verifier, and a rollback rule (Stern et al., 2018; Leviathan et al., 2023; Chen et al., 2023; Xia et al., 2023). The rest of this report develops that answer under one guiding theme: *branch prediction meets token prediction*. The goal is not only to summarize three papers, but to show how OS-style speculation principles reappear in modern LLM inference.

1.3 Why this topic fits Advanced Operating Systems

Although speculative decoding is usually introduced as a machine learning inference technique, its core logic is deeply systems-oriented.

- It resembles branch prediction: execute a predicted path early, then validate it and either commit or flush.
- It relies on *parallel verification* to reduce the number of expensive serialized commits, just as branch prediction relies on keeping the instruction pipeline full.
- Its real value depends on wall-clock latency, resource utilization, rollback overhead, and prediction quality rather than on algorithmic elegance alone.
- It transforms a strict sequential process into a pipeline with controlled speculation, bounded recovery cost, and better hardware utilization.

Therefore, speculative decoding is an appropriate course-paper topic for analyzing how algorithmic structure and runtime execution co-design each other.

1.4 From branch prediction to speculative decoding

The course presentation summarized the parallel between CPU speculation and LLM decoding as a four-way correspondence: the bottleneck is *control hazards* in CPUs and *serial decoding* in LLMs; the predictor is a *branch predictor* in CPUs and a *draft model* in LLMs; the verifier is the *resolved execution result* in CPUs and the *target model* in LLMs; and the recovery action is either *pipeline flush* or *draft rejection and correction*. This mapping provides a useful lens for the rest of the paper.

- A correct speculative guess should incur only small overhead and preserve continuity.

- A wrong guess should trigger rollback, but the rollback penalty must still be cheaper than fully serialized execution.
- The target model must verify several drafted positions with latency close to one normal generation step; otherwise the speculative path does not amortize its own overhead.

This branch-prediction lens explains why the final success criterion is not simply “better acceptance rate.” The real question is whether speculation reduces *stalls* in an expensive sequential process after accounting for verification and recovery cost.

Table 1.1: Formal correspondence between CPU branch prediction and LLM speculative decoding.

Dimension	CPU branch prediction	LLM speculative decoding
Bottleneck	Control hazard in an instruction pipeline	Serial dependency in autoregressive token commitment
Cheap predictor	Branch predictor predicts taken/not-taken direction and future path	Draft model predicts future tokens or hidden features
Expensive authority	Actual execution result resolves the correct path	Target model verifies candidate continuations
Useful speculative work	Instructions on the predicted path continue executing before branch resolution	Multiple future positions are drafted before token-by-token commitment
Commit condition	Predicted path matches resolved control flow	Drafted prefix passes target-model verification
Recovery action	Flush the wrong-path pipeline and restart from the correct PC	Reject the invalid suffix and resume generation from the first mismatch
Cost model	Predictor cost + misprediction penalty + recovered throughput	Draft cost + verifier cost + rollback penalty + accepted tokens per round
Success criterion	Pipeline continuity and higher effective IPC	Fewer serialized decoding rounds and lower end-to-end latency

1.5 Research objective and central thesis

This paper studies three representative works on speculative generation and asks one unifying question:

How do operating-systems speculation principles reappear in modern LLM inference, and how did speculative decoding evolve from an early greedy-equivalent decoding trick into a mature systems design space?

The central thesis is that the three selected papers correspond to three successive bottlenecks:

1. **Can multiple future positions be proposed and verified together?**
2. **How should the drafter and verifier be designed so that speedup remains practically meaningful?**
3. **If token-level drafting is too hard, what internal representation should be speculated instead?**

1.6 Contributions of this course paper

This paper makes two concrete contributions.

1. It corrects an important conceptual confusion: *SpecDec* (Xia et al., 2023) is treated here as a quality-preserving but not lossless relaxed-verification framework, whereas *EAGLE* (Li et al., 2024) is discussed as a distribution-preserving speculative sampling method.
2. It interprets all three papers from a systems viewpoint, emphasizing rollback, acceptance horizon, verifier cost, and deployment trade-offs.

Chapter 2

Background, Related Work, and Evaluation Framework

2.1 A unified draft–verify–commit view

The three selected papers, together with later speculative decoding and speculative sampling formulations, can be placed under a common draft–verify–commit execution template (Stern et al., 2018; Leviathan et al., 2023; Chen et al., 2023; Xia et al., 2023; Li et al., 2024):

1. **Draft.** A lightweight mechanism proposes candidate future states for the next K positions.
2. **Verify.** The original model evaluates those proposals in parallel or near-parallel form.
3. **Commit.** The system accepts a valid prefix, discards the invalid suffix, and resumes from the first rejected position.

The design space is therefore defined by four questions: what is drafted, how verification is implemented, what correctness guarantee is preserved, and how expensive rollback becomes.

2.2 Speculation as controlled rollback

The analogy to speculative execution in computer architecture is useful, but it should be stated carefully.

- In a CPU, speculation predicts future control flow; in decoding, speculation predicts future tokens or features.
- In a CPU, misprediction flushes the pipeline; in decoding, rejection discards a drafted suffix.
- In both cases, the value of speculation depends on the asymmetry between cheap prediction and expensive serialized commitment.

This analogy is not merely rhetorical. It explains why speculative decoding is fundamentally about *latency hiding under correctness constraints*.

2.3 Correctness regimes

The three papers differ sharply in what they preserve.

1. **Greedy equivalence.** Blockwise Decoding (Stern et al., 2018) preserves the output of a greedy base model because it commits only tokens that exactly match the base model’s own greedy choice.
2. **Quality-controlled approximation.** SpecDec (Xia et al., 2023) broadens acceptance with top- β and score-gap rules. It improves the speed–quality frontier, but it should not be described as exact target-distribution preservation.
3. **Lossless speculative sampling.** EAGLE (Li et al., 2024) preserves the target model’s generation distribution through target-model verification and correction, while changing only the drafting space.

This distinction is central to a rigorous comparison. Without it, one risks conflating fundamentally different algorithmic guarantees.

2.4 Latency model and acceptance horizon

For speculative decoding, a simple performance model is more informative than a purely algorithmic description. Let L be the output length, τ be the average accepted tokens per round, t_d be draft time per round, and t_v be verification time per round. Then the end-to-end latency can be approximated as

$$T \approx \frac{L}{\tau}(t_d + t_v). \quad (2.1)$$

Equation (2.1) explains the main trade-off that reappears in all three papers:

- larger drafting horizons can increase τ ,
- but larger horizons also increase t_d and may lower later-position acceptance,
- so empirical speedup is typically non-monotonic in the block size or draft length.

2.5 Evaluation dimensions

Following the writing requirements, this paper evaluates the three works across multiple dimensions instead of one summary metric only.

1. **Mechanism.** How is speculation constructed?
2. **Formal guarantee.** What notion of correctness is preserved?
3. **System overhead.** What extra model components are required?

4. **Experimental evidence.** What quantitative speedups and quality trends are reported?
5. **Boundary conditions.** When does the method become less effective?

Chapter 3

Paper I: Blockwise Parallel Decoding for Deep Autoregressive Models

3.1 Motivation and historical significance

Stern et al. (2018) provide the earliest clean speculative decoding skeleton for modern neural autoregressive models. Their key observation is that *generation is sequential, but validation can be parallel*. This opens the possibility of replacing one-token-per-round decoding with multi-token proposal followed by longest-prefix acceptance.

The paper’s historical significance is best understood through the OS analogy. It is the first work in this line to make the branch-prediction-style asymmetry explicit: *proposal can be speculative, but commit must be authoritative*. In other words, the system is allowed to guess multiple future tokens aggressively, but acceptance must still be decided by the base model. This is precisely the same logic by which a processor may speculatively execute instructions but may commit architectural state only after validation.

3.2 Method formulation

Let p_1 denote the base scoring model and p_i for $i = 2, \dots, K$ denote auxiliary future-position predictors. The baseline greedy decision at step $j + 1$ is

$$\hat{y}_{j+1} = \arg \max_y p_1(y \mid \hat{y}_{\leq j}, x). \quad (3.1)$$

The blockwise proposal stage predicts multiple future tokens in parallel:

$$\hat{y}_{j+i} = \arg \max_y p_i(y \mid \hat{y}_{\leq j}, x), \quad i = 1, \dots, K. \quad (3.2)$$

The verifier then computes the longest acceptable prefix length

$$\hat{k} = \max \left\{ n \in \{1, \dots, K\} : \forall i \leq n, \hat{y}_{j+i} = \arg \max_y p_1(y \mid \hat{y}_{\leq j+i-1}, x) \right\}. \quad (3.3)$$

If a mismatch occurs at position $j + i$, all later proposals in the same block are discarded because their contexts are no longer valid.

3.3 Framework and architectural realization

Figure 3.1 shows the three-stage workflow: parallel proposal, parallel verification, and longest-prefix acceptance. The design is historically important because it already separates *proposal* from *commitment*, even though the proposal mechanism still remains tightly attached to the base model.

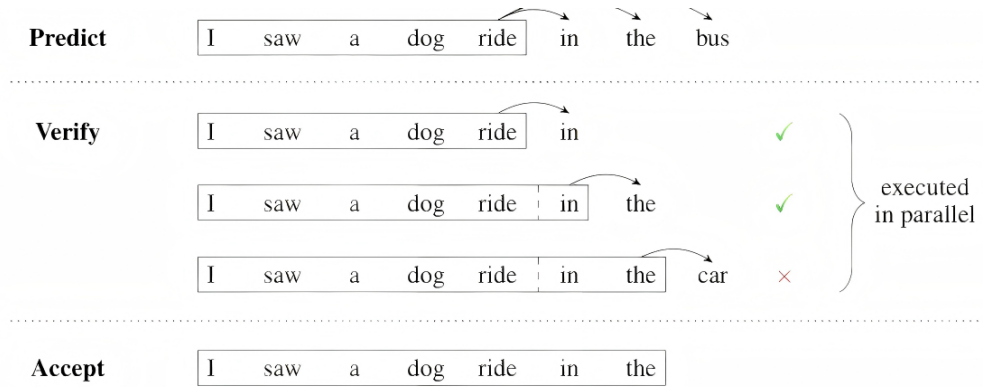


Figure 3.1: Blockwise parallel decoding pipeline: predict a block of future tokens, verify them with the base model, and commit only the longest valid prefix.

The paper also emphasizes an engineering point that remains relevant today: theoretical iteration savings alone do not guarantee latency savings. To reduce overhead, the authors merge the current round’s verification with the next round’s proposal, as illustrated in Figure 3.2.

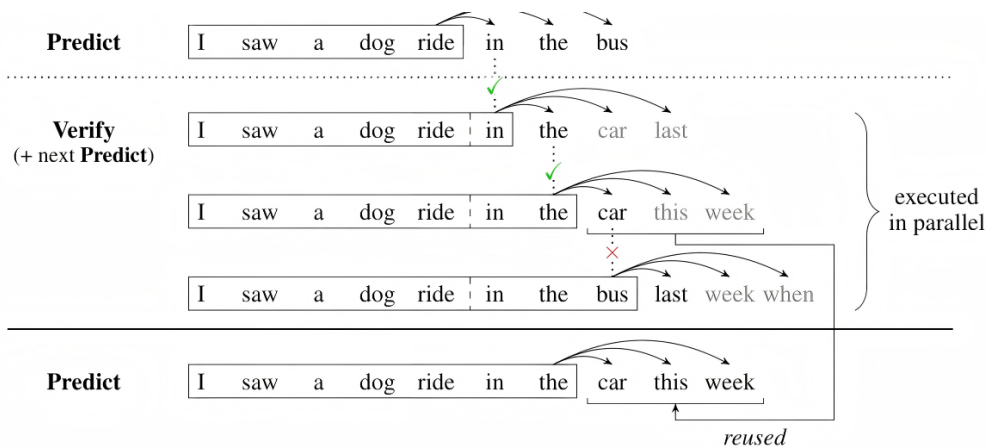


Figure 3.2: Combined scoring and proposal reduces the overhead of running separate proposal and verification passes.

For Transformer models, the proposal path is implemented by adding multiple future-token prediction heads on top of the decoder representation, shown in Figure 3.3. This makes the

method lighter than training several large independent models, but it also limits the expressiveness of long-horizon prediction.

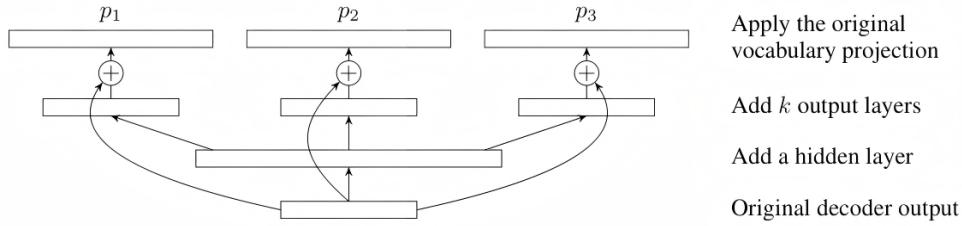


Figure 3.3: Transformer realization of Blockwise Decoding using shared backbone features and multiple future-token heads.

3.4 Quantitative evidence

The experimental results already expose the core speculative-decoding trade-off: larger blocks can reduce the number of rounds, but the benefit saturates because later proposals become harder to verify correctly. Table 3.1 summarizes the machine translation evidence that is most useful for this paper.

Table 3.1: Representative machine-translation results from Blockwise Decoding. Each cell reports BLEU / mean accepted block size.

K	Regular	Distillation	Fine Tuning	Both	Comment
2	25.81 / 1.51	26.52 / 1.55	25.74 / 1.78	26.58 / 1.88	conservative gain
4	25.84 / 1.73	26.31 / 1.85	25.05 / 2.69	26.36 / 3.27	better acceptance
6	26.08 / 1.76	26.26 / 1.90	24.69 / 2.98	26.18 / 4.18	quality trade-off appears
8	25.82 / 1.76	26.25 / 1.91	24.27 / 3.01	26.11 / 4.69	near best speedup
10	25.69 / 1.74	26.34 / 1.90	23.51 / 2.87	25.60 / 4.95	horizon begins to saturate

Table 3.2 highlights the distinction between accepted length and real latency. Mean accepted block size is not equal to wall-clock speedup because proposal and verification still incur overhead.

Table 3.2: Iteration reduction and wall-clock speedup reported by Blockwise Decoding.

Task	Best K	Mean accepted block size	Wall-clock speedup
Machine translation	8	4.7	3.31×
Super-resolution	6	5.3	4.0×

Figure 3.4 further shows the non-monotonic speedup behavior with increasing block size.

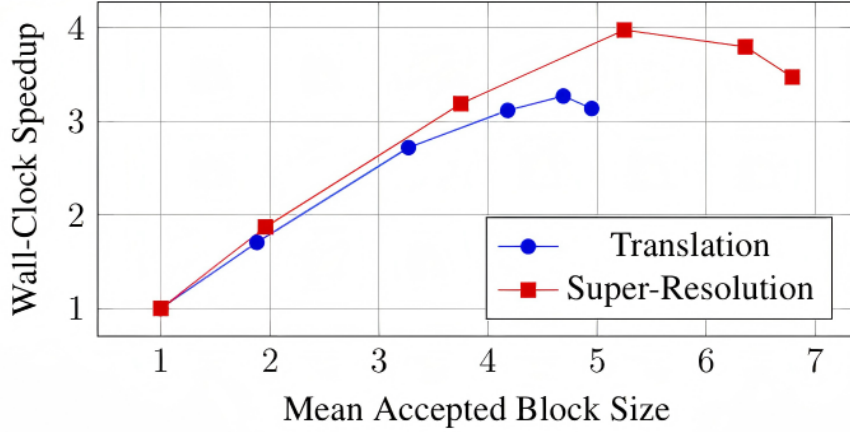


Figure 3.4: Wall-clock speedup is non-monotonic in the block size because larger horizons increase both prediction difficulty and overhead.

3.5 Strengths, limitations, and insight

Blockwise Decoding establishes three enduring principles.

1. Exactness can come from verification rather than from perfect proposal quality.
2. Accepted prefix length is only a proxy; system speed must be measured directly.
3. Proposal horizons help only if they remain cheap and sufficiently accurate.

From an OS perspective, the most important lesson is that accepted block size is analogous to *useful speculative depth*, while wall-clock speedup is analogous to *effective pipeline throughput*. A long speculative window is valuable only when the verification and rollback costs remain bounded. This is exactly why Figure 3.4 matters: it shows that more aggressive speculation does not guarantee higher end-to-end performance.

Its limitations also define the next research step. The proposal heads are tightly coupled to the base model, the method primarily supports greedy-style verification, and the gains saturate quickly as K grows. These weaknesses motivate the search for a better drafter and a more flexible verification rule.

Chapter 4

Paper II: Speculative Decoding for Accelerating Seq2seq Generation

4.1 Why Blockwise Decoding was not enough

Xia et al. Xia et al. (2023) inherit the same high-level execution template but argue that Blockwise Decoding underperforms for two reasons. First, head-based future-token prediction is too weak as a drafter. Second, strict exact-match verification rejects too many useful candidates. Their contribution is therefore not simply another speculative algorithm; it is a *better design of both draft and verification stages*.

This paper should be read as the point where speculative decoding becomes a *systems design problem* rather than a one-shot algorithmic trick. The core insight from the reading notes is that the bottleneck is not merely “can we guess multiple tokens,” but “can we guess them accurately enough and verify them without being overly conservative.” In OS terms, it is the difference between having speculation in principle and having a speculation mechanism whose predictor quality and recovery policy are both tuned for real throughput.

4.2 Spec-Drafter as a dedicated proposal model

Instead of attaching several heads to the original decoder, SpecDec trains a dedicated encoder–decoder drafter that predicts a block of masked target positions in parallel. Starting from the current accepted prefix $\hat{y}_{\leq j}$ and source input x , the drafter proposes

$$\tilde{y}_{j+1:j+K}. \quad (4.1)$$

The paper explicitly optimizes for two design principles.

1. **Capability principle:** the drafter must be accurate enough to make long accepted prefixes likely.
2. **Latency principle:** the drafter must remain lightweight enough that drafting cost does

not eliminate the gain.

These two principles directly instantiate Equation (2.1). Increasing accepted tokens per round improves the numerator of useful progress, but if draft latency becomes too large, the overall speedup collapses. This is why the paper’s contribution is not simply “a better model”: it is a predictor deliberately balanced for the speculation workload. Figure 4.1 contrasts Blockwise Decoding with the stronger Spec-Drafter.

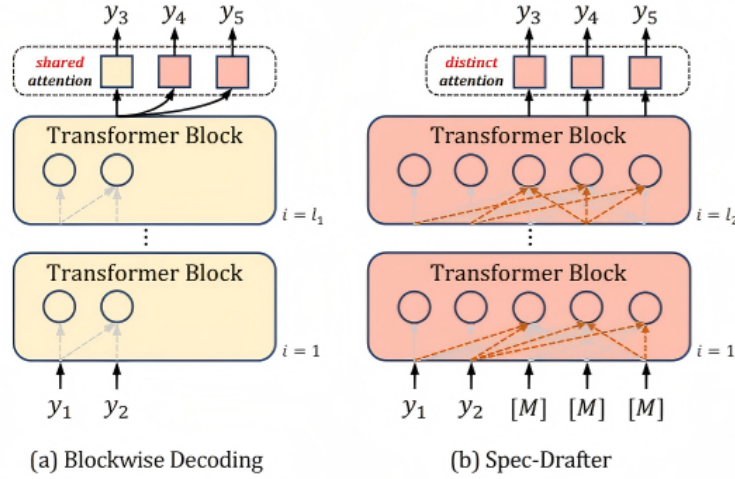


Figure 4.1: SpecDec replaces head-based future prediction with a dedicated drafter optimized for the block drafting task itself.

The analysis notes also emphasize that later draft positions should not share identical attention semantics. Figure 4.2 shows the intuition: a drafter specialized for multi-position prediction can allocate more appropriate attention patterns than simple future-token heads.

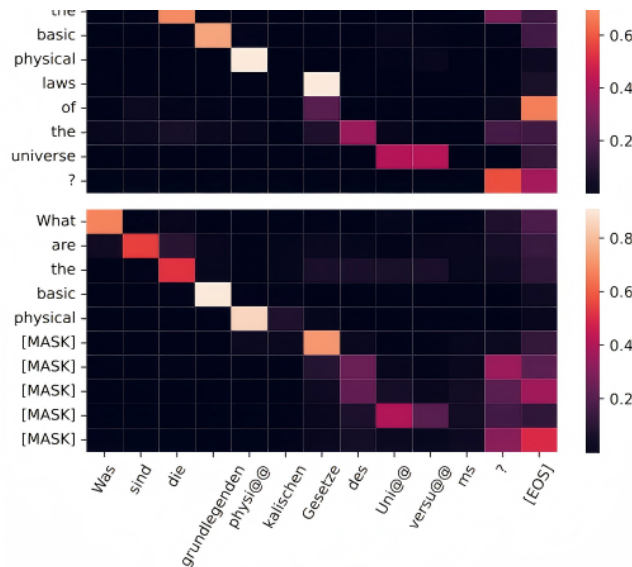


Figure 4.2: Distinct attention patterns across drafted positions motivate a specialized Spec-Drafter rather than a shared-head extension.

4.3 Relaxed Spec-Verification

SpecDec’s second core contribution is to loosen verification. Given the verification context

$$\Delta = (\hat{y}_{\leq j}, \tilde{y}_{j+1:j+i-1}, x), \quad (4.2)$$

the drafted token $\tilde{y}_{j+i}$ is accepted if it lies within a top- β candidate set of the autoregressive verifier and if its score gap to the verifier’s top-1 choice stays within threshold τ :

$$\log P(\tilde{y}_{j+i} \mid \Delta; \theta_{\text{AR}}) \geq \log P(\hat{y}_{j+i}^{(\beta)} \mid \Delta; \theta_{\text{AR}}), \quad (4.3)$$

$$\log P(\hat{y}_{j+i}^{(1)} \mid \Delta; \theta_{\text{AR}}) - \log P(\tilde{y}_{j+i} \mid \Delta; \theta_{\text{AR}}) \leq \tau. \quad (4.4)$$

Importantly, the acceptance test remains left-to-right: position $j + i$ is considered only if all previous drafted tokens in the same block have already been accepted. Figure 4.3 visualizes this relaxed verification logic.

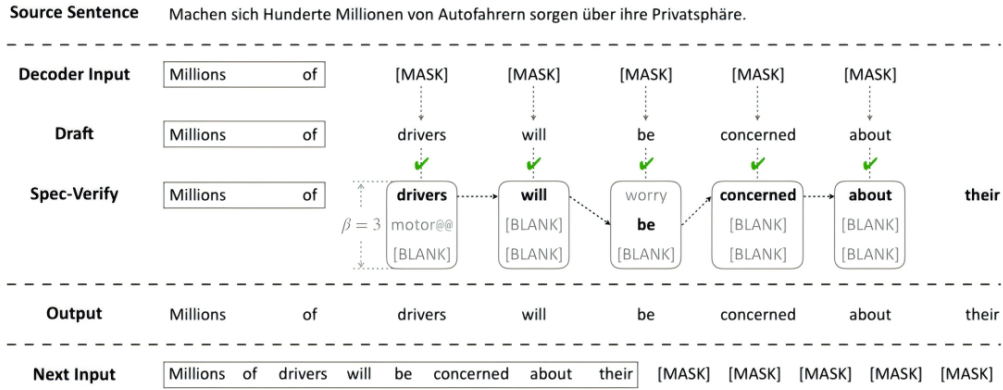


Figure 4.3: Spec-Verification accepts drafted tokens that are sufficiently competitive under the verifier, rather than insisting on exact top-1 equality.

4.4 Why the paper is practically important

SpecDec is best understood as the bridge from conceptual prototype to strong practical recipe.

- It accelerates the original autoregressive model instead of replacing it with a non-autoregressive alternative.
- It shows that proposal quality and proposal latency must be optimized jointly.
- It demonstrates that a relaxed verification rule can deliver a much better speed–quality frontier than strict exact matching.

However, this also means that SpecDec should be described as a *quality-controlled approximate* framework, not as exact target-distribution preservation. In OS language, it relaxes the commit

policy slightly in order to reduce false rejections of useful speculative work. The resulting gain is not “free correctness,” but a better global operating point on the latency–fidelity curve.

4.5 Experimental analysis

Table 4.1 summarizes the main translation results. The central empirical message is that SpecDec scales to larger draft lengths more gracefully than Blockwise Decoding.

Table 4.1: Main translation results of SpecDec versus baselines. Each cell reports speedup / BLEU.

Method	EN→DE	DE→EN	EN→RO	RO→EN
Transformer-base (beam)	1.0× / 28.89	1.0× / 32.53	1.0× / 34.96	1.0× / 34.86
Transformer-base (greedy)	1.1× / 28.73	1.1× / 32.18	1.1× / 34.83	1.1× / 34.65
Blockwise ($K = 10$)	1.9× / 28.73	2.0× / 32.18	1.4× / 34.83	1.4× / 34.65
Blockwise ($K = 25$)	1.6× / 28.73	1.7× / 32.18	1.2× / 34.83	1.2× / 34.65
SpecDec ($K = 10$)	4.2× / 28.90	4.6× / 32.61	3.9× / 35.29	4.1× / 34.88
SpecDec ($K = 25$)	5.1× / 28.93	5.5× / 32.55	4.6× / 35.45	4.8× / 35.03

Table 4.2 shows why the stronger drafter matters. A head-based drafter yields only 1.7× speedup, whereas the full Spec-Drafter reaches 5.1×.

Table 4.2: Drafter ablation in SpecDec.

Model	Tok.	BLEU	t_d	Speed
Transformer-base (beam)	1.00	28.89	–	1.0×
Head-based drafter	2.32	28.02	0.81	1.7×
Spec-Drafter w/o Principle I	7.05	28.56	2.29	4.4×
Spec-Drafter w/o Principle II	8.21	28.95	10.87	4.0×
Full Spec-Drafter	8.23	28.93	5.21	5.1×

The verification hyperparameters also reveal a controllable trade-off. Table 4.3 shows that more relaxed settings increase accepted length and speed, but can reduce BLEU.

Table 4.3: Verification trade-off in SpecDec. Each cell reports Tok. / BLEU, with speed shown in parentheses when provided.

τ	Top-3 ($\beta = 3$)	Top-5 ($\beta = 5$)
1	7.89 / 26.97 (5.0 $\times$)	7.92 / 26.88
2	8.75 / 26.84	8.83 / 26.79
3	9.51 / 26.71	9.64 / 26.68
4	10.11 / 26.60	10.63 / 26.59
5	10.46 / 26.58	11.01 / 26.58 (6.8 $\times$)

Finally, Figure 4.4 shows that latency and throughput must be considered together rather than separately.

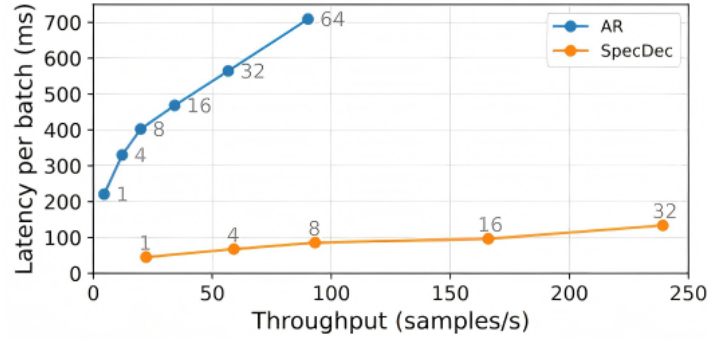


Figure 4.4: SpecDec improves both latency and throughput, but the operating point still depends on drafter cost and acceptance behavior.

4.6 System interpretation and limits

SpecDec’s major insight is that the main optimization target is not proposal accuracy alone but end-to-end latency under verifier constraints. This is why Equation (2.1) is more explanatory than any single acceptance metric. The reading notes repeatedly emphasize a systems-level conclusion: SpecDec is especially attractive in *memory-rich*, *latency-critical*, *low-batch* settings, where simply increasing batch size is not an acceptable substitute for reducing single-request latency.

This is exactly the kind of trade-off an OS course should care about. High throughput can often be improved by coarser batching, but latency-sensitive online services care about tail delay and fast completion of individual requests. SpecDec matters because it attempts to improve both throughput and latency without replacing the original model. The main limitations are equally important:

- an extra drafter must still be trained and stored;

- the best settings depend on K , β , and τ ;
- the quality guarantee is empirical and controlled, not exact;
- the strongest benefits appear in latency-sensitive, not arbitrarily high-batch, regimes.

These constraints motivate the next question: if token-level drafting remains difficult, should the system speculate in a different representation space?

Chapter 5

Paper III: EAGLE and Feature-Level Speculative Sampling

5.1 Motivation: token prediction may be the wrong interface

Li et al. Li et al. (2024) argue that token-level speculative drafting becomes increasingly difficult for modern LLMs. Discrete token trajectories branch sharply, and a lightweight token drafter may fail to maintain sufficiently high acceptance. EAGLE therefore challenges the default interface of speculation itself: rather than drafting tokens directly, it drafts hidden features close to the language-model head.

This shift can be interpreted as a systems redesign of the predictor interface. Blockwise Decoding and SpecDec both assume that the right object to speculate is the next token sequence. EAGLE argues that this interface is too unstable: the prediction target branches too sharply, so the speculative window collapses. In OS terms, the paper is not merely building a better predictor; it is redefining the *state representation* in which prediction becomes easier.

5.2 Feature uncertainty as the real bottleneck

EAGLE’s most important conceptual contribution is not merely “features are smoother than tokens.” The deeper claim is that future features are *uncertain* unless the next sampled token is known. Figure 5.1 illustrates this point.

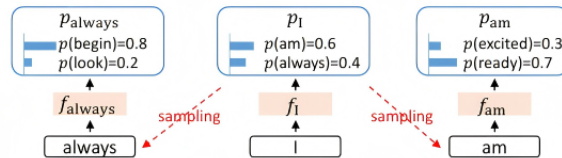


Figure 5.1: Feature uncertainty: future hidden features are not uniquely determined by past features alone because the next token decision changes the latent trajectory.

This diagnosis changes the research question from “how to build a better token drafter” to “what additional information is needed to make feature extrapolation well-posed.”

5.3 Method: shifted tokens plus feature autoregression

Let $\mathbf{f}_i$ denote the target model’s hidden feature near the penultimate layer. EAGLE drafts future features from the input pair $(F_{1:i}, T_{2:i+1})$, where the token sequence is shifted by one step so that feature prediction is anchored by the next token context. The regression objective is

$$\mathcal{L}_{\text{reg}} = \text{SmoothL1}(f_{i+1}, \text{DraftModel}(T_{2:i+1}, F_{1:i})) . \quad (5.1)$$

To ensure that predicted features remain useful for actual token generation, the target model’s LM head is reused:

$$p_{i+2} = \text{Softmax}(\text{LMHead}(f_{i+1})), \quad (5.2)$$

$$\hat{p}_{i+2} = \text{Softmax}(\text{LMHead}(\hat{f}_{i+1})). \quad (5.3)$$

The classification loss is

$$\mathcal{L}_{\text{cls}} = \text{CE}(p_{i+2}, \hat{p}_{i+2}), \quad (5.4)$$

and the final training objective becomes

$$\mathcal{L} = \mathcal{L}_{\text{reg}} + w_{\text{cls}} \mathcal{L}_{\text{cls}}, \quad w_{\text{cls}} = 0.1. \quad (5.5)$$

The predictor is therefore trained to be accurate both in feature space and in the token distribution induced by those features.

This dual-objective formulation is important. EAGLE does not treat features as arbitrary hidden variables; it constrains them as *execution-relevant latent states* whose downstream token distribution must remain useful to the verifier. That is the key reason the method remains compatible with lossless target-side verification instead of becoming a purely heuristic latent extrapolation scheme.

5.4 Framework and verification pipeline

Figure 5.2 situates EAGLE relative to token-level alternatives, while Figure 5.3 shows the full feature-drafting pipeline.

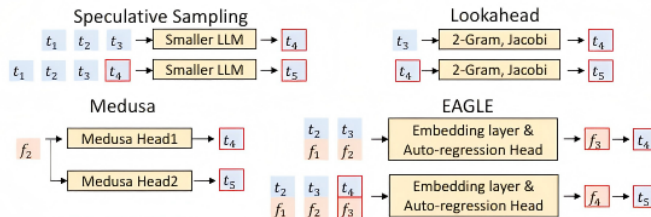


Figure 5.2: Method comparison: EAGLE differs from classical token drafting by predicting future features while using shifted tokens to resolve uncertainty.

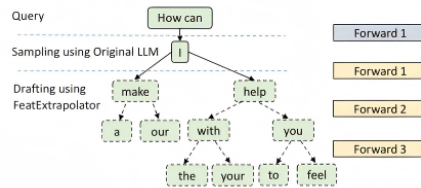


Figure 5.3: EAGLE pipeline: feature drafting, LM-head token generation, draft-tree expansion, and tree-attention verification by the target LLM.

The target model still performs the authoritative verification, typically through tree attention so that multiple drafted candidates can be checked in one pass. This is why EAGLE can change the drafting interface while keeping the generation semantics of the target model unchanged.

Here the OS analogy becomes especially strong. Tree attention is the mechanism that amortizes verification across many speculative branches, much as a well-designed speculative execution engine must validate useful work with less cost than fully serial execution. EAGLE’s contribution is therefore twofold: a better prediction space and a verifier organization that can exploit that space efficiently.

5.5 Quantitative results

Figure 5.4 reports the overall speedup picture. Across dense LLMs and tasks, EAGLE typically achieves around $2.7\times$ – $3.8\times$ speedup, outperforming earlier token-level baselines in the reported settings.

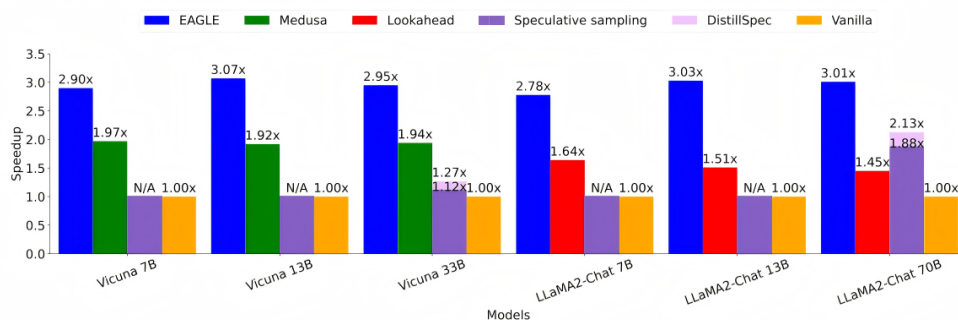


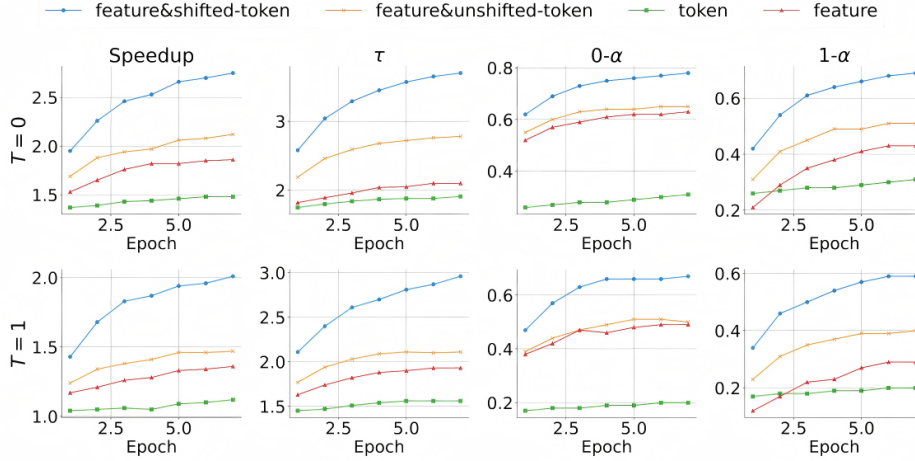
Figure 5.4: Greedy-decoding speedup reported for EAGLE across multiple LLMs.

Table 5.1 summarizes several headline results extracted from the reading notes.

Table 5.1: Representative EAGLE results reported in the analysis notes.

Model / Task	Speedup	Average accepted length τ
Vicuna 7B / HumanEval	3.33×	4.29
Vicuna 13B / HumanEval	3.58×	4.39
LLaMA2-Chat 13B / HumanEval	3.76×	4.52
LLaMA2-Chat 70B / HumanEval	3.52×	4.42
Vicuna 7B / GSM8K	3.01×	4.00
Vicuna 13B / Alpaca	3.03×	3.95

The ablation in Figure 5.5 is especially important because it directly validates the paper’s conceptual claim: feature drafting alone is not enough; *feature drafting with shifted tokens* is the robust design.

**Figure 5.5:** Input ablation in EAGLE: shifted-token conditioning materially improves acceptance and speed by resolving feature uncertainty.

The reading notes further highlight several practical observations.

- Tree attention improves the accepted length by roughly 0.6–0.8 and speedup by roughly 0.3–0.5 over chain-style verification.
- EAGLE composes well with aggressive runtime stacks such as `gpt-fast`, improving throughput from 55.1 tok/s to 100.2 tok/s in FP16 and from 106.9 tok/s to 160.4 tok/s in int4.
- Performance is weaker on MoE models such as Mixtral, where the reported speedup is closer to 1.5×

These observations are valuable because they show that EAGLE is not just a modeling paper. It also exposes the deployment boundary conditions of speculative decoding: batch size, model architecture, and verifier structure all materially affect realized speedup.

5.6 Strengths and limitations

EAGLE’s main strengths are conceptual and practical at the same time.

1. It identifies a deeper failure mode of token-level speculation: feature uncertainty.
2. It changes the drafting space to a representation that is easier to extrapolate.
3. It preserves target-model semantics through target-side verification while remaining deployment-oriented.

From an OS perspective, the paper’s most important insight is that good speculation depends not only on predictor accuracy but also on *predicting in the right state space*. If the visible interface is too discontinuous, no lightweight predictor will scale well. EAGLE’s feature-level drafting is therefore analogous to moving speculation to a representation where future state transitions are smoother and rollback becomes less frequent.

Its limitations are also instructive.

- It still requires a nontrivial trained drafter head.
- It depends more tightly on target-model internals than token-only methods.
- It is strongest for dense-model latency optimization and less universally strong for MoE or very high-batch settings.

Chapter 6

Cross-Paper Comparison and Mechanism-Level Discussion

6.1 One topic, three bottlenecks

The three papers should not be treated as direct substitutes for one another. Each attacks a different bottleneck along the same line of development.

1. **Blockwise Decoding:** can multiple future positions be proposed and verified jointly?
2. **SpecDec:** how should the drafter and verifier be redesigned to improve practical speedup in seq2seq generation?
3. **EAGLE:** if token drafting is intrinsically hard for LLMs, what better prediction interface exists?

6.2 Comparative summary

Table 6.1 provides a condensed comparison.

Table 6.1: Cross-paper comparison of mechanisms, guarantees, and bottlenecks.

Paper	Draft object	Verification rule	Guarantee	Reported speedup
<i>Blockwise Decoding</i> (Stern et al., 2018)	Future tokens via auxiliary heads	Longest-prefix greedy match	Exact greedy equivalence	up to $3.31\times$ MT, $4.0\times$ super-resolution
<i>SpecDec</i> (Xia et al., 2023)	Future tokens via dedicated drafter	Top- β + score-gap acceptance	Quality-controlled approximation	$4.2\times$ – $5.5\times$ on translation, $5.1\times$ on summarization

Paper	Draft object	Verification rule	Guarantee	Reported speedup
<i>EAGLE</i> (Li et al., 2024)	Future hidden features with shifted tokens	Tree-based target-model speculative sampling	Lossless target-distribution preservation	roughly 2.7×–3.8× on dense LLMs

6.3 How the drafting interface evolves

The most important evolution is not simply that the reported speedup improves. Instead, the object being speculated changes.

- Blockwise Decoding speculates *future tokens* using shallow future-offset heads.
- SpecDec still speculates *future tokens*, but does so with a stronger dedicated drafter and a less rigid verifier.
- EAGLE speculates *future features* and uses shifted tokens only to disambiguate latent-state evolution.

This progression shows a common systems-design pattern: when one interface becomes too difficult to optimize, later work may succeed by moving the optimization problem to a more stable internal representation.

6.4 Correctness, rollback, and efficiency

From a systems perspective, speculative decoding can be analyzed using three tightly coupled notions.

1. **Correctness rule.** What condition makes a speculative step safe?
2. **Rollback granularity.** How much work is discarded after rejection?
3. **Verifier amortization.** How many future positions can one expensive verification pass cover?

Blockwise Decoding uses a strict correctness rule and therefore has simple rollback semantics. SpecDec relaxes correctness in exchange for a better throughput frontier. EAGLE preserves correctness while moving the proposal mechanism to a space with higher predictability. In all cases, the main optimization goal is to maximize verifier amortization without letting rollback become too frequent.

6.5 Which paper contributes what to the final understanding

If these papers are arranged as a pedagogical sequence, their roles are complementary rather than redundant.

- **Blockwise Decoding** contributes the earliest algorithmic skeleton and the longest-prefix acceptance principle.
- **SpecDec** contributes the first truly strong practical recipe for seq2seq generation and clarifies that a better drafter is often more valuable than a larger horizon alone.
- **EAGLE** contributes the representation-level insight that token prediction may not be the best drafting target for LLM-scale acceleration.

6.6 Open questions

Even with these advances, speculative decoding still faces several unresolved issues.

1. How should speculation adapt to batch size and serving mode?
2. How much model-specific internal access is acceptable before portability becomes too poor?
3. Can verifier-aware training produce better acceptance without significantly enlarging the drafter?
4. How should speculative methods behave on MoE architectures, multimodal models, and extremely long contexts?

These questions indicate that future progress will require joint work across algorithms, model design, and runtime systems.

Chapter 7

Conclusion

This paper analyzed three representative speculative decoding papers under a unified thesis-style framework and argued that they form a clear research trajectory rather than a loose collection of acceleration tricks. *Blockwise Decoding* (Stern et al., 2018) established the foundational execution pattern of parallel proposal followed by authoritative verification and longest-prefix commitment. *SpecDec* (Xia et al., 2023) demonstrated that the practical quality of speculation depends heavily on the drafter and on verification policy, and that useful acceleration comes from optimizing end-to-end latency rather than proposal accuracy alone. *EAGLE* (Li et al., 2024) showed that the next major leap comes from changing the drafting interface itself: future hidden features can be easier to extrapolate than future discrete tokens, provided that feature uncertainty is resolved with shifted-token conditioning.

The main insight for an Advanced Operating Systems course is that speculative decoding is not merely an LLM trick; it is an instance of a much older systems principle. Like branch prediction, it exists because a sequential dependency threatens throughput. Like branch prediction, it works only when three conditions are satisfied: the predictor is cheaper than the serialized stall it avoids, the verifier can validate speculative work efficiently, and the rollback penalty is bounded enough that speculation still pays on average. Under this lens, the three papers can be read as progressively better answers to the same systems question: how should one choose the predictor, the verification rule, and the speculative state representation so that useful work dominates wasted work?

The broader lesson is therefore that speculative decoding is best viewed as a systems problem with an algorithmic surface. Its performance depends not only on model quality but also on acceptance horizon, rollback cost, verifier amortization, batching regime, and hardware mapping. For the speculative decoding topic as a course-paper subject, this makes it especially valuable: it sits exactly at the intersection of probabilistic modeling, runtime execution, and operating-systems style speculation. In the strongest summary of the report, *branch prediction meets token prediction*: speculative decoding offers a principled way to convert a strictly serialized generation loop into a controlled speculative pipeline whose success depends on how well one balances correctness, predictor cost, and amortized parallel work.

Bibliography

- Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise Parallel Decoding for Deep Autoregressive Models. In *Advances in Neural Information Processing Systems*, 2018.
- Heming Xia, Tao Ge, Peiyi Wang, Si-Qing Chen, Furu Wei, and Zhifang Sui. Speculative Decoding: Exploiting Speculative Execution for Accelerating Seq2seq Generation. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Proceedings of the International Conference on Machine Learning*, 2024.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding. In *Findings of the Association for Computational Linguistics: ACL*, 2024.
- Zixuan Zhou, Xuefei Ning, Ke Hong, Tianchen Fu, Jiaming Xu, Shiyao Li, Yixin Lou, Lining Wang, Zhihang Yuan, Xiuhong Li, Shengen Yan, Guohao Dai, Xiao-Ping Wang, Yu Dong, and Yu Wang. A Survey on Efficient Inference for Large Language Models. *arXiv preprint arXiv:2404.14294*, 2024.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 6th edition, 2019.